

Multithreading in Java

1. What is a Thread?

A **thread** is a small unit of execution within a program.

In simple words:

A thread is an independent path of execution inside a program.

For example, a music player application may perform several tasks simultaneously:

- Play music
- Download a song
- Display animation
- Respond to user input

Each task can be handled by a separate thread.

Process vs Thread

Process	Thread
A program in execution	A part of a process
Requires more resources	Requires fewer resources
Independent memory space	Shares memory with other threads
Creation is slower	Creation is faster
Example: Chrome, VS Code	Downloading, printing, animation

2. What is Multithreading?

Multithreading is the process of executing multiple threads concurrently within a single program.

Example

Suppose a Java application has three tasks:

```
Thread 1 → Download file  
Thread 2 → Play music  
Thread 3 → Display progress
```

All three threads can make progress during the same period.

Advantages of Multithreading

1. **Better CPU utilization**
2. **Faster execution of independent tasks**
3. **Improved application responsiveness**
4. **Resource sharing**
5. Useful for games, servers, animations, networking, etc.

3. Creating a Thread in Java

There are two basic approaches commonly taught in Core Java:

Method 1: Extending the `Thread` class

```
class MyThread extends Thread {  
  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}  
  
public class Demo {  
    public static void main(String[] args) {  
  
        MyThread t1 = new MyThread();  
  
        t1.start();  
    }  
}
```

Important Point

```
t1.start();
```

starts a **new thread** and internally causes the `run()` method to execute.

Do **not** normally call:

```
t1.run();
```

because that simply executes `run()` like an ordinary method in the current thread.

4. Method 2: Implementing Runnable

A class can implement the `Runnable` interface.

```
class MyTask implements Runnable {  
  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}  
  
public class Demo {  
    public static void main(String[] args) {  
  
        MyTask task = new MyTask();  
  
        Thread t1 = new Thread(task);  
  
        t1.start();  
    }  
}
```

Why use Runnable?

Java supports **single inheritance**, so if our class already extends another class, we cannot extend `Thread`.

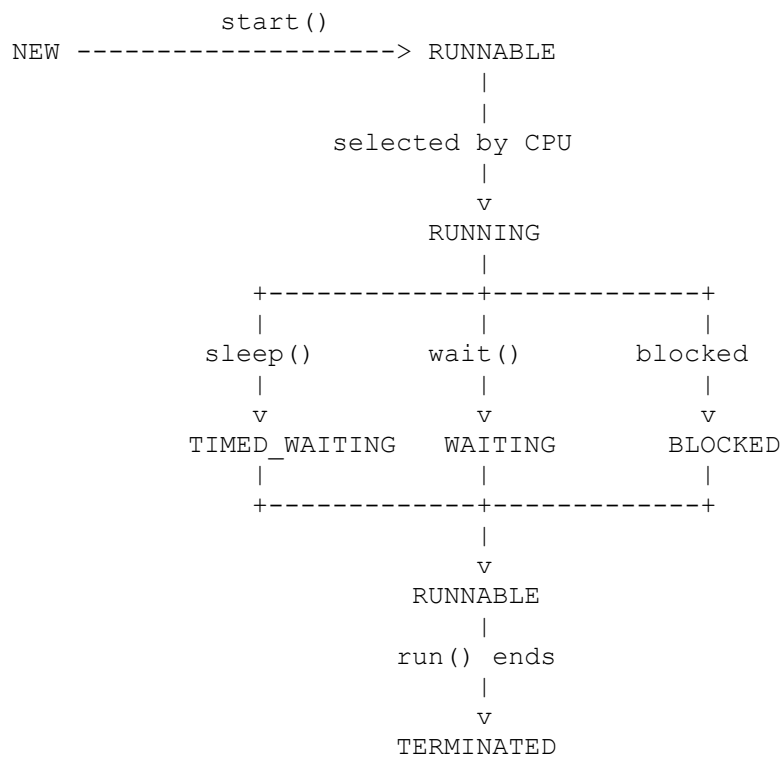
Therefore, `Runnable` provides a more flexible approach.

5. Java Thread Lifecycle

A thread does not remain in the same state throughout its execution.

It passes through different states.

Basic lifecycle



In Java's official `Thread.State` enumeration, **RUNNING is not a separate state**. A running thread is represented by **RUNNABLE**.

6. Different Thread States

1. NEW

Thread object has been created but `start()` has not been called.

```
Thread t = new Thread();
```

State:

NEW

2. RUNNABLE

After calling:

```
t.start();
```

the thread becomes eligible to run.

NEW → RUNNABLE

The JVM scheduler decides when the thread gets CPU time.

3. BLOCKED

A thread enters the **BLOCKED** state when it is waiting to acquire a monitor lock.

Example:

```
synchronized void display() {  
    // code  
}
```

If another thread already owns the lock, the waiting thread becomes blocked.

4. WAITING

A thread enters the **WAITING** state when it waits indefinitely for another thread to perform some action.

Common example:

```
wait();
```

It can also occur with methods such as:

```
join();
```

under appropriate conditions.

5. TIMED_WAITING

The thread waits for a specified period.

Example:

```
Thread.sleep(2000);
```

The thread waits for approximately **2 seconds** before becoming eligible to run again.

Other examples include timed versions of `wait()` and `join()`.

6. TERMINATED

When the `run()` method finishes, the thread has completed execution.

RUNNABLE → TERMINATED

A terminated thread cannot be restarted.

7. Simple Thread Lifecycle Example

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
  
        try {  
            Thread.sleep(2000);  
        }  
        catch (InterruptedException e) {  
            System.out.println(e);  
        }  
  
        System.out.println("Thread completed");  
    }  
}  
  
public class Demo {  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread();  
  
        System.out.println(t1.getState());  
  
        t1.start();  
  
        System.out.println(t1.getState());  
    }  
}
```

Possible output:

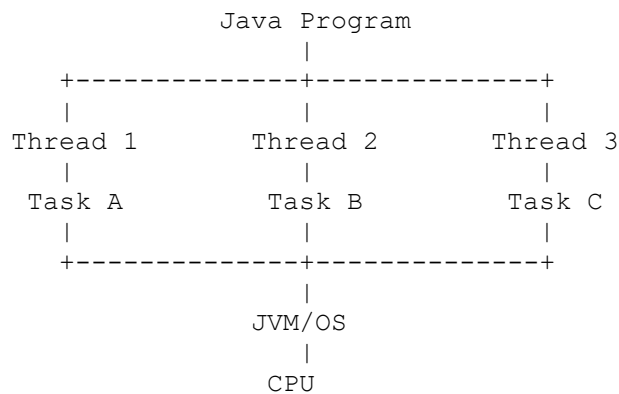
```
NEW  
RUNNABLE  
Thread is running  
Thread completed
```

The exact observed state can vary because thread scheduling is controlled by the JVM and operating system.

8. Java Thread Model

The **Java Thread Model** describes how Java manages multiple threads and how those threads execute within a program.

A simple representation is:



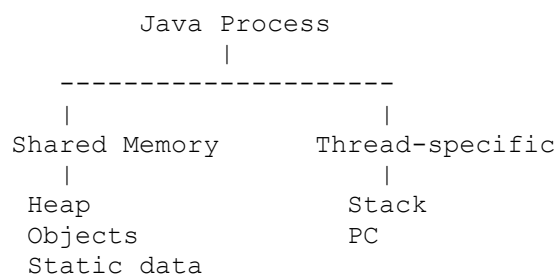
All threads belonging to the same process generally share resources such as:

- Heap memory
- Objects
- Static variables

But each thread has its own:

- Stack
- Program counter
- Execution path

Memory concept



This sharing makes communication between threads easier, but it can also create problems such as **race conditions**.

9. Main Thread

Every Java application starts with a special thread called the **main thread**.

For example:

```
public static void main(String[] args) {  
    }  
}
```

The `main()` method is executed by the main thread.

We can check its name:

```
public class Demo {  
    public static void main(String[] args) {  
        System.out.println(  
            Thread.currentThread().getName()  
        );  
    }  
}
```

Output:

```
main
```

10. Creating Multiple Threads

```
class Task extends Thread {  
  
    public void run() {  
  
        for(int i = 1; i <= 5; i++) {  
            System.out.println(  
                Thread.currentThread().getName()  
                + " : " + i  
            );  
        }  
    }  
}  
  
public class Demo {  
  
    public static void main(String[] args) {  
  
        Task t1 = new Task();  
        Task t2 = new Task();  
  
        t1.start();  
        t2.start();  
    }  
}
```

Possible output:

```
Thread-0 : 1  
Thread-1 : 1  
Thread-0 : 2  
Thread-1 : 2  
Thread-1 : 3  
Thread-0 : 3  
...
```

Important observation

The output order is **not guaranteed**.

The JVM/operating-system scheduler decides which runnable thread gets CPU time.

11. Important Thread Methods

Method	Purpose
<code>start()</code>	Starts a new thread
<code>run()</code>	Contains the task performed by the thread
<code>sleep()</code>	Pauses execution for a specified time
<code>join()</code>	Waits for another thread to complete
<code>getName()</code>	Returns thread name
<code>setName()</code>	Sets thread name
<code>getPriority()</code>	Gets thread priority
<code>setPriority()</code>	Sets thread priority
<code>currentThread()</code>	Returns currently executing thread
<code>getState()</code>	Returns current thread state
<code>isAlive()</code>	Checks whether thread is still alive

12. `start()` vs `run()`

This is an important exam question.

`start()`

```
t1.start();
```

- Creates/initiates a new thread of execution.
- JVM eventually invokes `run()` on that thread.

`run()`

```
t1.run();
```

- Does not create a new thread.
- Executes like a normal method call in the current thread.

Remember

Use `start()` to start a thread.

13. Simple Real-Life Example

Imagine a food-delivery application.

Main Application

```
|
+---- Thread 1 → Process customer order
|
+---- Thread 2 → Track delivery
|
+---- Thread 3 → Send notification
|
+---- Thread 4 → Update payment status
```

Instead of making the application wait for one task to finish before starting another, multiple tasks can make progress concurrently.

14. Key Points for Students

- A **thread** is an independent path of execution.
- **Multithreading** means executing multiple threads concurrently.
- Java provides built-in support for multithreading.
- Threads can be created using `Thread` or `Runnable`.
- `start()` should be used to start a thread.
- `run()` contains the thread's task.
- Every Java application starts with a **main thread**.
- Threads share the heap but have their own stacks.
- Java defines six official thread states:
NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
- Thread scheduling is generally controlled by the JVM and underlying operating system.
- Multiple threads improve responsiveness and resource utilization, but shared data can introduce synchronization problems.